

Stateful Architecture

1. What is Stateful Architecture?

A **stateful architecture** means the server **stores client state/data between requests**.

The server remembers previous interactions.

Examples of stored state:

None

Login session

Shopping cart

User preferences

Temporary workflow data

Chat connection state

2. Core Idea

If user sends first request:

None

Login successful

Server stores:

None

Session ID

Authenticated user info

Next request depends on previously stored state.

3. Example

Suppose 3 servers exist:

None

Server 1

Server 2

Server 3

User A logs in on Server 1.

Then:

None

Session stored on Server 1

All future requests from User A should go to Server 1.

4. Why This Matters

If next request goes to Server 2:

None

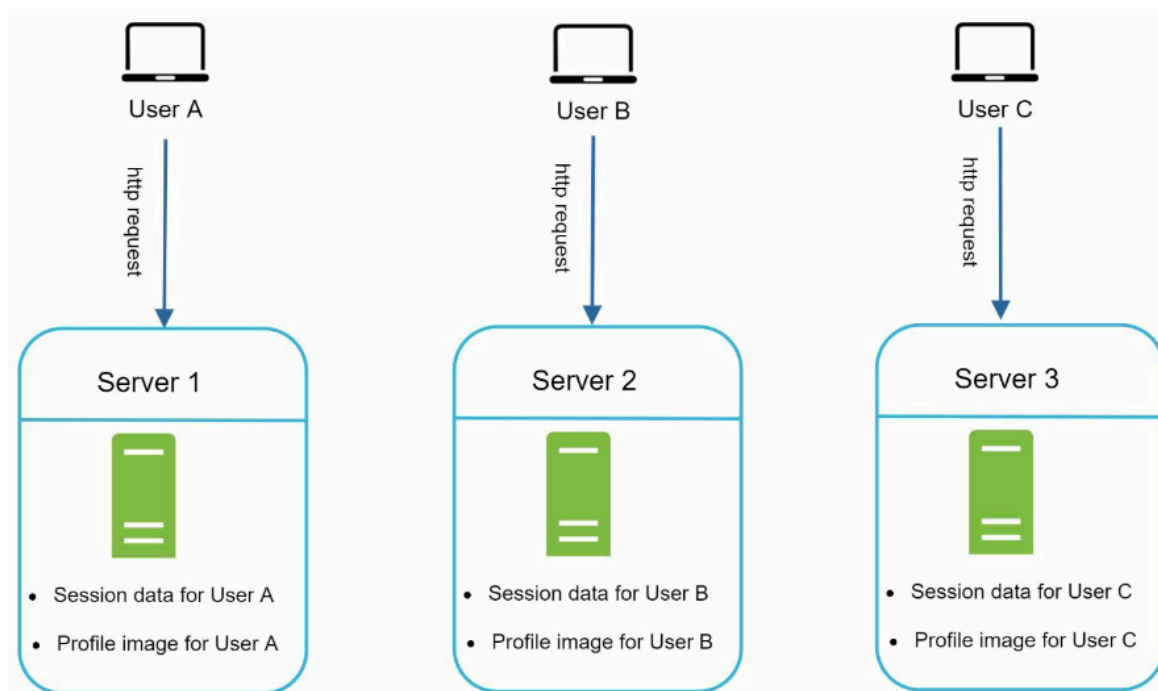
No session found

Authentication fails

User logged out

Because Server 2 does not have User A's state.

5. Typical Diagram



None

User A → Server 1

User B → Server 2

User C → Server 3

Each user tied to one server.

6. Sticky Sessions

Load balancer must route same user repeatedly to same server.

Called:

None

Sticky Session

Session Affinity

Persistent Routing

7. Example Sticky Session Flow

None

Login request → Server 1

Dashboard request → Server 1

Checkout request → Server 1

Profile request → Server 1

8. Advantages of Stateful Design

None

Simple session handling

Fast access to in-memory user state

Good for real-time connections

Easy temporary workflows

9. Disadvantages

None

Hard to scale horizontally

Needs sticky sessions

Server failure loses sessions

Uneven traffic distribution

Hard deployments

Complex failover

10. Server Failure Problem

If Server 1 crashes:

None

All users mapped to Server 1 lose session

Need re-login

State lost

11. Scaling Problem

Add new Server 4:

None

Existing users still attached to old servers

New server underused initially

Rebalancing difficult

12. Uneven Load Example

None

Server 1 = 10,000 users

Server 2 = 2,000 users

Server 3 = 1,500 users

Some servers overloaded.

13. Common Stateful Systems

Use stateful model in:

None

WebSocket servers

Game servers

Video conferencing

Live collaboration tools

Legacy web sessions

14. Stateful vs Stateless

Feature	Stateful	Stateless
Stores session	Yes	No
Easy scaling	Harder	Easier
Sticky routing	Usually needed	No

Failover	Harder	Easier
Real-time connection apps	Strong fit	Limited

15. How Modern Systems Solve It

Instead of storing state in server memory:

None

Store sessions in Redis

Store auth in JWT

Use shared database

Use distributed cache

Then servers become more stateless.

16. Example with Redis

None

User logs in

↓

Session stored in Redis

↓

Any server can validate session

Better scalability.

17. Good Use Cases

Use stateful architecture when:

None

Connection itself is stateful

Low-latency session memory needed

Game room state required

Real-time socket sessions

18. Bad Use Cases

Avoid for:

None

Large web APIs

Microservices at scale

Auto-scaling cloud apps

High availability stateless systems

19. Interview Talking Points

If stateful chosen, mention:

None

Sticky sessions required

Replication of state needed

Failover strategy needed

Load imbalance risk

Session storage externalization

20. Real Example

Legacy login system:

None

Apache server stores PHP sessions locally

Same user must hit same machine

Classic stateful web architecture.

21. Better Modern Pattern

None

Stateless app servers

Shared Redis session store

Load balancer routes anywhere

Best of both worlds.

22. One-Line Summary

Stateful architecture means the server remembers client data between requests, so future requests often must return to the same server, making scaling and failover more complex.